

HPPS Assignment A4

Björn Petterson (zts900), Manar(chs380), Thomas Andersen (kgq325)

April 15, 2026

Introduction

Comparatively, this assignment felt much more manageable than the previous one. For this assignment we talked about the different skills it takes reading code versus writing code. Implementing the functions once we had a sense that we understood what was in the handout was relatively easy. While we are pretty confident that our implementations work as intended, function pointers still seem like a mystery.

Specific questions

Evaluate the temporal and spatial locality of the four programs you have implemented.

For both `id_query_naive.c` and `coord_query_naive.c` we have bad spatial locality. While records are stored in contiguous memory with a constant stride, the ids that we are actually iterating through are not. To test this we temporarily added a print statement to the loop in the lookup function that printed out the memory addresses of each record as it was compared against the `needle`. The stride is 200 bytes, while we would expect it to be `sizeof(int64_t)` for optimized spatial locality. The temporal locality in `id_query_naive.c` is also poor. Due to the program uses brute force to iterate through all records for each query, every record is accessed once per query with no reuse of cache lines. This results in frequent cache misses and inefficient memory access patterns.

The temporal locality of `coord_query_naive.c` is poor as well, each query involves a linear scan through the dataset, which results in bit reuse of previously accessed data. Therefore we have the same situation as before, we the program access each record per

query.

Due to adding an index, we improve both spatial and temporal locality in `id_query_naive`. The index is stored in a separate array, allowing ID's to be accessed with a much smaller stride. This benefits the spatial locality by ensuring that the fields are stored together in contiguous memory. The temporal locality is fairly improved because of the indexed data structure being reused across queries.

In `id_query_binsort.c`, where we sort the records by ID and use binary search, we improve both spatial and temporal locality. Sorting ensures that the data is accessed in a structured way during the search, which reduces the memory stride and aligns with cache line boundaries, and therefore improves spatial locality. The temporal locality is also improved due to that the binary search repeatedly accesses nodes closer to the middle of the dataset, which are probable remained in cache for comparisons. The structured reuse of cache lines minimizes memory latency and improves query performance.

Do your programs corrupt memory? Do they leak memory? How do you know?

We have tested the following implementation files for memory leaks by running `clang` with the `-fsanitize=address` flag which got the following result:

- `id_query_naive.c`: No reported leaks (malloc: nano zone abandoned)
- `coord_query_naive.c`: No reported leaks (malloc: nano zone abandoned)
- `id_query_indexed.c`: No reported leaks (malloc: nano zone abandoned)
- `id_query_binsort.c`: No reported leaks (malloc: nano zone abandoned)

How confident are you that your programs are correct?

We are reasonably confident that our programs work as intended. We have tested each of them manually using both hits and misses for ids as well as various coordinates. During benchmarking we have piped in files with 10,000 line of ids and locations without error. One thing that one might consider for these are creating graceful exits for the loops, so that a function can be called by a user and not forcing the user to `ctrl+c` to stop. This is easily implemented in its most rudimentary form: If the value from `stdin` matches something like 'exit' or 'quit' then exit the program. We did not implement this, as the

primary use for functions such as these did not seem to be manually prompting a user for single look-ups in the terminal.

Benchmark your programs on various workloads and explain the differences. How did you pick the workloads?

Workload 1

For the first workload we created a test file of 10,000 valid ids created using `random_ids` from a file of 20000 records. We benchmarked using hyperfine with 10 warm-up rounds. It's worth noting that using `random_ids` to create test ids we do not get any misses as every single id in the test batch exists in the file we're testing against. For the naive version that means that worst case doesn't happen which in turn means that It was surprising

Command	Mean [ms]	Min [ms]	Max [ms]	Relative
<code>id_query_naive</code>	129.5 ± 1.9	126.9	132.2	5.95 ± 0.58
<code>id_query_binsort</code>	21.8 ± 2.1	20.8	43.0	1.00
<code>id_query_indexed</code>	123.6 ± 3.2	120.1	127.3	5.68 ± 0.56

Table 1: Performance comparison of different query methods, no missing ids

that `id_query_indexed` did not perform better

Workload 2

For the second workload we did what we did for workload 1, but we increased each number by 4 to introduce misses.

Command	Mean [ms]	Min [ms]	Max [ms]	Relative
<code>id_query_naive</code>	220.7 ± 4.1	216.3	228.8	11.56 ± 0.37
<code>id_query_binsort</code>	19.1 ± 0.5	18.6	22.7	1.00
<code>id_query_indexed</code>	205.4 ± 6.0	198.7	212.1	10.76 ± 0.42

Table 2: Performance comparison of different query methods, includes missing ids

We see both indexed and naive performing worse, but binary performing better. Intuitively that makes sense because the algorithm handles basically the same for hits and misses.

Workload 3

Let's add more benchmarks on a different computer.

Overall, we can conclude that binsort is the fastest algorithm, which aligns with its empirically faster $O(\log(n))$ time complexity since we halve our search area each time, compared to the $O(n)$ search complexity of the other algorithms, where, in the worst case (a miss), we have to look at n elements. We can also observe that binsort excels in cases where we handle more misses due to this characteristic.

Surprisingly, the indexed method does not perform significantly better than the naive approach. The idea was that managing only a list of IDs and pointers during the search would be faster than handling the whole dataset, particularly due to the use of malloc to create new arrays with better memory locality. This makes me think that we may have made some mistakes in the implementation or that other factors are at play. Perhaps the overhead of constructing and managing the index lists is significant enough to reduce performance. One would expect it to significantly outperform the naive approach in cases with many queries.

For a large dataset of 100k records, we see binsort excel again. We observe more similar computational times for fewer indexes compared to many, likely because there is some fixed "cost" associated with indexing and reading arrays into memory, etc.

Table 3: Performance comparison of different query methods across benchmarks

Benchmark	Algo	Mean Time (ms)	Relative
20,000 records, valid IDs	Naive	770.5 ± 38.2	1.22x slower
	Indexed	895.3 ± 247.5	1.42x slower
	Binsort	630.7 ± 327.6	1.00x
20,000 records, offset IDs	Naive	1079.0 ± 92.0	2.38x slower
	Indexed	1025.0 ± 280.0	2.26x slower
	Binsort	452.9 ± 113.4	1.00x
100,000 records, 10 valid IDs	Naive	1244.0 ± 27.0	1.01x slower
	Indexed	1235.0 ± 43.0	1.00x
	Binsort	1235.0 ± 35.0	1.00x
100,000 records, 1,000 valid IDs	Naive	1584.0 ± 57.0	1.30x slower
	Indexed	1332.0 ± 65.0	1.09x slower
	Binsort	1221.0 ± 33.0	1.00x